complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + nonematch() negative validation

filter() null checks + nonematch() negative validation

filter() comparisons + nonematch() negative validation

filter() method calls returning boolean + nonematch() negative validation

distinct() rely on equals() / hashcode() + nonematch() negative validation

distinct() immutable objects + nonematch() negative validation

map() transform objects + nonematch() negative validation

map() create new objects + nonematch() negative validation

map() call pure functions + nonematch() negative validation

flatmap() return stream + nonematch() negative validation

flatmap() split data + nonematch() negative validation

flatmap() expand collections + nonematch() negative validation

sorted() natural ordering + nonematch() negative validation

sorted() custom comparator + nonematch() negative validation

sorted() multi-field compare + nonematch() negative validation

peek() logging + nonematch() negative validation

peek() debugging + nonematch() negative validation

peek() metrics + nonematch() negative validation

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = noneMatch()**
- Purpose = **negative validation** (assert *no element* violates a rule)
- **Short-circuiting**: stops on the first violation
- Java **8+**, no collectors/reduce

---

1 filter() boolean checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterBooleanNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean ok = true;

      for (Integer n : list) {
         if (n > 0) {
            if (n % 2 != 0) {                                    // noneMatch condition
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

2 filter() null checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterNullNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean ok = true;

      for (String s : list) {
         if (s != null) {
            if (s.length() == 0) {                                  // isEmpty()
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

3 filter() comparisons + noneMatch() — Negative validation

```
import java.util.*;

public class FilterComparisonNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(25, 30, 40);

      boolean ok = true;

      for (Integer age : list) {
         if (age >= 18) {
            if (age > 60) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

4 filter() method calls returning boolean + noneMatch() — Negative validation

```
import java.util.*;

public class FilterMethodNoneMatch {

   static class User {
      boolean banned;
      User(boolean banned) { this.banned = banned; }
      boolean isBanned() { return banned; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(false),
            new User(false)
      );

      boolean ok = true;

      for (User u : list) {
         if (u.isBanned()) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

5 distinct() rely on equals() / hashCode() + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctEqualsNoneMatch {

   static class Person {
      String email;

      Person(String email) { this.email = email; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return email.equals(p.email);
      }

      public int hashCode() {
         return email.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("a@mail.com"),
            new Person("a@mail.com"),
            new Person("b@mail.com")
      );

      Set<Person> unique = new HashSet<Person>(list);

      boolean ok = true;

      for (Person p : unique) {
         if (p.email.endsWith("@spam.com")) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

6 distinct() immutable objects + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctImmutableNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");

      Set<String> unique = new HashSet<String>(list);

      boolean ok = true;

      for (String lang : unique) {
         if (lang.length() < 3) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

7 map() transform objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapTransformNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean ok = true;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (upper.indexOf(" ") != -1) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

8 map() create new objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapCreateObjectNoneMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean ok = true;

      for (String name : list) {
         UserDTO dto = new UserDTO(name);
         if (dto.name.trim().length() == 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

9 map() call pure functions + noneMatch() — Negative validation

```
import java.util.*;

public class MapPureFunctionNoneMatch {

   static int normalize(int n) {
      return n * 10;
   }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean ok = true;

      for (Integer n : list) {
         int value = normalize(n);
         if (value <= 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

10 flatMap() return Stream + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapStreamNoneMatch {
   public static void main(String[] args) {

      List<List<Integer>> list = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4)
      );

      boolean ok = true;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n < 0) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

11 flatMap() split data + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapSplitNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");

      boolean ok = true;

      for (String s : list) {
         String[] words = s.split(" ");
         for (int i = 0; i < words.length; i++) {
            if (words[i].length() < 3) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

12 flatMap() expand collections + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapExpandNoneMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean ok = true;

      for (List<String> values : map.values()) {
         for (String s : values) {
            if (s.length() == 0) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

13 sorted() natural ordering + noneMatch() — Negative validation

```
import java.util.*;

public class SortedNaturalNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(
            Arrays.asList(1, 2, 3)
      );

      Collections.sort(list);

      boolean ok = true;

      for (Integer n : list) {
         if (n <= 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

14 sorted() custom comparator + noneMatch() — Negative validation

```
import java.util.*;

public class SortedCustomNoneMatch {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit")
      );

      Collections.sort(list, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                                    // reverse
         }
      });

      boolean ok = true;

      for (String name : list) {
         if (name.length() < 4) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

15 sorted() multi-field compare + noneMatch() — Negative validation

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0)
               return nameCompare;
            return e1.age - e2.age;
         }
      });

      boolean ok = true;

      for (Employee e : list) {
         if (e.age < 18) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

16 peek() logging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekLoggingNoneMatch {

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean ok = true;

      for (Integer n : list) {

         // peek equivalent (logging)
         System.out.println("Checking: " + n);

         // noneMatch condition
         if (n < 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

17 peek() debugging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekDebugNoneMatch {

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      boolean ok = true;

      for (Integer n : list) {

         // peek equivalent (debugging)
         if (n == 20) {
            System.out.println("Debug hit");
         }

         // noneMatch condition
         if (n < 10) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

18 peek() metrics + noneMatch() — Negative validation

```
import java.util.*;

public class PeekMetricsNoneMatch {

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);

      int seen = 0;                                    // AtomicInteger replaced with simple counter
      boolean ok = true;

      for (Integer n : list) {

         // peek equivalent (metrics)
         seen++;

         // noneMatch condition
         if (n <= 0) {
            ok = false;
            break;
         }
      }

      System.out.println("OK: " + ok);
      System.out.println("Processed: " + seen);
   }
}
```